PRE-SEED · OCTOBER 2026

INDEX0

Next-generation code review: the verification layer for AI-written code.

A working product, a published audit of our own claims, and no benchmark we can't reproduce.

01 — THE PROBLEM

AI made code cheap. Knowing it's correct stayed expensive.

10×

more security findings from AI-assisted developers, who also wrote 3–4× more code.

45%

of AI-generated samples introduced an OWASP Top 10 flaw, flat across model generations.

66%

of developers say the frustration is code that is "almost right, but not quite".

The bottleneck moved from **writing** code to **proving** it. Every team reviews more code, faster, with the same senior eyes.

Apiiro, Sep 2025 · Veracode, Jul 2025 · Stack Overflow Developer Survey 2025

02 — THE SOLUTION

A gate, not another generator.

TIER 1

Architect

Turns the objective into invariants that can be violated.

TIER 2

Developer

Edits the real files. No hunk arithmetic, no stubs.

TIER 3

Critic

Adversarial review. Only critical/error findings can block.

TIER 4

QA

Runs the candidate code against the repo's suite in a network-isolated container.

EVIDENCE The exit code decides — not a model's opinion of the diff.

SIGNED Every cycle is recorded, hash-chained and attributable to a key.

YOURS Your cloud, your model keys, your source never leaves.

services/agent-orchestrator/src/graph/review_loop.py · services/sandbox-manager · packages/gnap

03 — HOW IT WORKS

Four tiers, one verdict.

```
$ ./demo/run-demo.sh
==> litellm gateway (free route, no database)
    model groq reachable
==> sandbox-manager (provider=docker, image=node:22)
==> agent-orchestrator
==> running the review cycle

model      : groq
status     : qa_verified
iterations : 1
critic     : approve
qa passed  : True
tests      : 3 passed / 0 failed in 1332 ms
sandbox    : live_microvm (exit 0)
history
  architect   plan_proposal
  developer   implementation
  critic      review_verdict   approve
  qa          test_result      passed
```

The patch only counts because QA **executed** it — no network, read-only mount, 1.3 seconds. This is the whole diff it shipped.

```
--- a/src/math.js
+++ b/src/math.js
@@ -1,4 +1,7 @@
 export function divide(numerator, denominator) {
+  if (denominator === 0) {
+    throw new RangeError('division by zero');
+  }
   return numerator / denominator;
 }
```

demo/runs/20261031T002004Z — request, cycle transcript and diff committed with the code

04 — KEY DIFFERENTIATORS

Three things nobody else does.

WHOLE-FILE Models are bad at hunk arithmetic. We give file contents, derive the diff with `difflib`, and reject corrupt patches.

ADVISORY CRITIC `request_changes` with no critical/error finding is auto-upgraded to `approve`. The prompt forbids inventing requirements.

TEST-EDIT GUARD Candidate writes to test paths are dropped, so the assertions that judge the verdict are always the repository's own.

Discovered when a Gemini run edited test/math.test.js mid-demo. We grade our own homework on the repo's word, not the model's.

05 — PRODUCT

What ships today.

CLI + TUI Terminal agent with worktree isolation and GitHub Device Flow login.

4-TIER LOOP Architect, Developer, Critic, QA — LangGraph, 30/30 tests green.

SANDBOX Docker or Firecracker/E2B, `--network none`, read-only source mount.

ROUTING LiteLLM proxy: OpenAI, Anthropic, Azure, Groq, Gemini, or your own.

AUDIT GNAP commit signing, hash-chained cycles, attributable to a key.

06 — USE CASES

Where the gate pays for itself.

SECURITY AI fixes that compile and pass lint but introduce OWASP flaws — verified against the suite before merge.

REFACTORS Large mechanical changes where "looks right" is not evidence.

DEPENDENCY BUMPS Version upgrades that silently break call sites the reviewer never opens.

PLAUSIBLE CODE The 46% of AI output that is "almost right, but not quite".

07 — USER BENEFITS

What the buyer gets.

REPLAYABLE An artifact you can re-run offline and verify without trusting us.

IN-PERIMETER Source and inference stay in the customer's tenant, on their model contract.

AUDITABLE Hash-chained, attributable, readable by an auditor, not a vendor.

HONEST We published the 15/36 capabilities and deleted the benchmarks we couldn't reproduce.

No telemetry is collected — own-keys means we see nothing, which is the point and the limitation.

Boring on purpose.

ROUTING LiteLLM proxy — any OpenAI-compatible backend, including air-gapped.

ISOLATION Docker or Firecracker/E2B microVMs, `--network none`, read-only source mount.

COORDINATION GNAP: multi-agent consensus recorded in `.gnap/` inside Git history.

WORKTREES Each agent gets its own Git worktree; no shared-state collisions.

09 — MARKET SIZE

The budget is real. The estimates are soft, so we show the range.



AI coding tools, 26.2% CAGR. Two analyst houses differ by **~4×** — so we cite the range, not a point, and never quote a TAM for "AI code verification" because none exists.

Our budget line is **AI governance**: AppSec and audit, not the IDE.

Mordor Intelligence, 2026 · MarketsandMarkets · Gartner: 90% of enterprise engineers on AI code assistants by 2028

10 — LOOKING FORWARD

Three scenarios.

SCENARIO	OUTCOME	REVENUE
Best case	**Becomes the verification standard for regulated engineering orgs.**	**Market leader, $1B+ yearly**
Realistic	5% of the top 500 regulated companies; three design partners convert.	$20–30M ARR
Worst case	Remains a 10-engineer service in one regulated account; IP and audit trail survive.	Time-saver, not a company

We state the worst case because you will find it anyway. Mitigations are in the notes.

Numbers are illustrative, not forecast. We have no revenue and no users today.

Platform and AppSec, not the IDE budget.

WHO SIGNS Platform owns the engineering system; AppSec owns the finding backlog; CISO owns "where did that code go?".

WHERE 200–5,000 developers in finance, healthcare, government — already on Azure/Bedrock/on-prem contracts.

HOW Founder-led, three design partners, services-heavy deals, in the customer's tenant so we stay out of SOC 2 scope.

WHY NOW DORA, the EU Cloud Sovereignty Framework, India DPDP. Sovereignty is now a scored procurement criterion.

European Commission, 20 Oct 2025 · India MeitY, 13 Nov 2025 · Gartner, 9 Feb 2026

We audited our own claims and deleted the ones we couldn't reproduce.

15/36

claimed capabilities verifiable today, listed line by line in **PROOF_STATUS.md**.

0

published benchmark scores. The old 96.2% SWE-bench and 92% AiderBench numbers were unverified and removed.

30/30

orchestrator tests green, plus 14 sandbox-manager and the full monorepo build.

4

committed end-to-end runs: `qa_verified`, exit 0, 3–4 tests passed in 1.2–1.7 s.

"We are building a verification product. The first thing we verified was ourselves."

APPENDIX

The five things that can kill us.

01 **Verification is a feature, not a company.** Cursor, GitHub and Qodo can bundle review-plus-tests into seats they already own.

02 **No forcing function.** "Audit-ready evidence" is a nice-to-have until an auditor demands it.

03 **Two people cannot pass enterprise procurement.** SOC 2, pen tests, 9–18 month cycles — runway ends first.

04 **Credibility debt is real.** The audit found an OTP bypass, injectable telemetry and a review path that could return approvals offline. All fixed and pinned by tests.

05 **Own-keys means no telemetry.** No usage data, no expansion signal, no ROI proof we can show a board.

Stated plainly, because you will find them anyway.